# Mech Interp cheat sheet front · observe, intervene, decompose

**MICAI 2026** · Fri 6 Nov
tested: TL 3.9.0 · sae-lens 6.54.12 · sklearn 1.9.1

## Load a model, cache every activation NB1

```
import torch
from transformer_lens import HookedTransformer, utils
torch.set_grad_enabled(False)          # inference only
device = "cuda" if torch.cuda.is_available() else "cpu"
model = HookedTransformer.from_pretrained("gpt2-small",
                                          device=device)
prompt = "The Eiffel Tower is in the city of"
tokens = model.to_tokens(prompt)       # [1, pos], BOS first
model.to_str_tokens(prompt)  # ['<|endoftext|>', 'The', ...]
logits, cache = model.run_with_cache(tokens)
# logits: [batch, pos, d_vocab]
top = logits[0, -1].topk(5).indices
model.to_str_tokens(top)       # [' London', ' Paris', ...]
resid = cache["resid_post", 6]
# same tensor as cache["blocks.6.hook_resid_post"]
_, small = model.run_with_cache(   # cache only what you need
    tokens, names_filter="blocks.6.hook_resid_post")
```

## Hook names `utils.get_act_name(name, L)`

| Hook point | Shape | Meaning |
|---|---|---|
| `hook_embed` | [b, pos, d_model] | token embedding |
| `blocks.{L}.hook_resid_pre` | [b, pos, d_model] | stream into layer L |
| `blocks.{L}.hook_resid_mid` | [b, pos, d_model] | after attn, before MLP |
| `blocks.{L}.hook_resid_post` | [b, pos, d_model] | stream out of layer L |
| `blocks.{L}.attn.hook_pattern` | [b, head, q, k] | attention weights |
| `blocks.{L}.attn.hook_z` | [b, pos, head, d_head] | per-head output |
| `blocks.{L}.hook_attn_out` | [b, pos, d_model] | attn write to stream |
| `blocks.{L}.mlp.hook_post` | [b, pos, d_mlp] | MLP neurons |
| `blocks.{L}.hook_mlp_out` | [b, pos, d_model] | MLP write to stream |
| `ln_final.hook_normalized` | [b, pos, d_model] | input to unembed |

Short names: `utils.get_act_name("z", 9)` → `blocks.9.attn.hook_z`; `cache["pattern", 9]`. GPT-2 small: 12 layers, 12 heads, d_model 768, d_head 64.

## Logit lens: read any layer as a prediction NB1

```
for L in range(model.cfg.n_layers):
    r = cache["resid_post", L][:, -1:, :]   # last position
    lg = model.unembed(model.ln_final(r))   # [1, 1, d_vocab]
    tok = lg[0, -1].argmax().item()
    print(L, model.to_single_str_token(tok))
# all layers at once (final LayerNorm applied for you)
acc = cache.accumulated_resid(layer=-1, incl_mid=False,
                              apply_ln=True, pos_slice=-1)
lens_logits = acc @ model.W_U    # [n_layers+1, 1, d_vocab]
```

The lens is a *view*, not a cause: early layers were never trained to be read this way.

## Sparse autoencoders with SAELens NB3

```
from sae_lens import SAE
# sae-lens v6 returns the SAE itself (no tuple)
sae = SAE.from_pretrained(release="gpt2-small-res-jb",
                          sae_id="blocks.8.hook_resid_pre",
                          device=device)
hook = sae.cfg.metadata.hook_name  # 'blocks.8.hook_resid_pre'
text = "The Golden Gate Bridge is in San Francisco"
_, cache = model.run_with_cache(text)
x = cache[hook]                    # [1, pos, d_model]
f = sae.encode(x)                  # [1, pos, d_sae], sparse
x_hat = sae.decode(f)              # [1, pos, d_model]
vals, idx = f[0, -1].topk(5)       # top latents, last token
rel_err = ((x_hat - x).norm() / x.norm()).item()
sae.W_dec.shape                    # [d_sae, d_model]
```

Gemma Scope 2 in NB3 (Gemma 3 1B IT, float32 on T4): `SAE.from_pretrained("gemma-scope-2-1b-it-res", "layer_17_width_16k_l0_medium")`, layer 17 of 26. Without a GPU NB3 falls back to GPT-2 + `gpt2-small-res-jb` automatically.

## Activation patching: clean → corrupted NB2

```
from functools import partial
ctx = "When John and Mary went to the store,"
clean = ctx + " John gave a drink to"
corrupt = ctx + " Mary gave a drink to"
c_tok = model.to_tokens(clean)
x_tok = model.to_tokens(corrupt)       # must be same length
ans = model.to_single_token(" Mary")
wrong = model.to_single_token(" John")

def logit_diff(logits):        # > 0: model prefers " Mary"
    return (logits[0, -1, ans] - logits[0, -1, wrong]).item()

clean_logits, clean_cache = model.run_with_cache(c_tok)
CLEAN = logit_diff(clean_logits)
CORR = logit_diff(model(x_tok))

def patch_resid(act, hook, pos):  # hook(act, hook) -> act
    act[:, pos, :] = clean_cache[hook.name][:, pos, :]
    return act

name = utils.get_act_name("resid_pre", 8)
patched = model.run_with_hooks(
    x_tok, fwd_hooks=[(name, partial(patch_resid, pos=-1))])
recovered = (logit_diff(patched) - CORR) / (CLEAN - CORR)
# 0 = no effect, 1 = clean behaviour fully restored

def patch_head(z, hook, head):     # one head, via hook_z
    z[:, :, head, :] = clean_cache[hook.name][:, :, head, :]
    return z

z9 = utils.get_act_name("z", 9)
model.run_with_hooks(
    x_tok, fwd_hooks=[(z9, partial(patch_head, head=9))])

# persistent hook: stays until you remove it
model.add_hook(name, partial(patch_resid, pos=-1))
out = model(x_tok)
model.reset_hooks()
# scoped hook: removed automatically on exit
with model.hooks(fwd_hooks=[(name,
                             partial(patch_resid, pos=-1))]):
    out = model(x_tok)
```

```
# every (layer, position) of resid_pre in one call (slow on CPU)
from transformer_lens import patching
metric = lambda lg: lg[0, -1, ans] - lg[0, -1, wrong]
grid = patching.get_act_patch_resid_pre(
    model, x_tok, clean_cache, metric)    # [n_layers, pos]
```

**Pitfalls.** Clean and corrupted prompts must tokenize to the same length. A hook that edits in place must still `return` the tensor. A hook left by `add_hook` silently changes every later run: call `model.reset_hooks()`.

## Steering: add a latent's decoder direction NB3

```
i, alpha = int(idx[0]), 8.0      # latent index, strength
v = sae.W_dec[i]                 # [d_model]
def steer(act, hook):
    return act + alpha * v       # all positions
with model.hooks(fwd_hooks=[(hook, steer)]):
    print(model.generate("I think that", max_new_tokens=20,
                         do_sample=False, verbose=False))
```

Sweep `alpha` (0, 2, 4, 8, 16) and compare with a *random* direction of the same norm: steering vectors are not unique.



Turn over: probes, calibration, circuit tracing, glossary, links

# Mech Interp cheat sheet back · probe, audit, trace

Snippets continue from the front (same `model`, `utils`)
**circuit-tracer 0.5.0**: calls checked against the released source

## Linear probe on the residual stream NB3 · NB5

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
pos = ["I love this", "What a great day", "Amazing work",
       "Best meal ever", "I am so happy", "Truly delightful"]
neg = ["I hate this", "What an awful day", "Horrible work",
       "Worst meal ever", "I am so sad", "Truly disgusting"]
texts = pos + neg
y = np.array([1] * len(pos) + [0] * len(neg))
L = 6
hp = utils.get_act_name("resid_post", L)
def feats(t):                  # last-token residual at layer L
    _, c = model.run_with_cache(t, names_filter=hp)
    return c[hp][0, -1].float().cpu().numpy()
X = np.stack([feats(t) for t in texts])     # [n, d_model]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.33, stratify=y, random_state=0)
probe = LogisticRegression(max_iter=1000, C=1.0)
probe.fit(X_tr, y_tr)
p = probe.predict_proba(X_te)[:, 1]   # P(class 1)
acc = probe.score(X_te, y_te)
```

Compare with a baseline (shuffled labels, layer 0, bag of words). When transfer matters, split by *rule or source*, not by row.

## Calibration and selective prediction NB5

```
from sklearn.metrics import roc_auc_score, brier_score_loss
from sklearn.calibration import calibration_curve
# conf: P(verdict correct); correct: 1 if the judge was right
rng = np.random.default_rng(0)
conf = rng.uniform(0.5, 1, 200)
correct = (rng.uniform(size=200) < conf).astype(int)

auroc = roc_auc_score(correct, conf)     # 0.5 = chance
brier = brier_score_loss(correct, conf)  # lower is better

def ece(conf, correct, n_bins=10):  # expected calib. error
    ids = np.clip((conf * n_bins).astype(int), 0, n_bins - 1)
    gap = 0.0
    for b in range(n_bins):
        m = ids == b
        if m.any():
            d = abs(correct[m].mean() - conf[m].mean())
            gap += d * m.mean()
    return gap

# reliability diagram: observed accuracy per confidence bin
frac_ok, mean_conf = calibration_curve(correct, conf, n_bins=10)

order = np.argsort(-conf)                # risk-coverage
n = np.arange(1, len(conf) + 1)
coverage = n / len(conf)
risk = np.cumsum(1 - correct[order]) / n

t = 0.8                          # threshold policy
auto = conf >= t                 # auto-accept; rest -> human
esc = (1 - correct[auto]).mean()  # error among accepted
print(f"coverage {auto.mean():.2f}, error {esc:.2f}")
```

Choose `t` on a held-out calibration split, never on the test set. Report coverage and error among accepted together.

## Circuit tracing with circuit-tracer NB4

```
# fresh runtime; NB4 pins (nnsight 0.8 breaks 0.5.0):
# %pip install circuit-tracer==0.5.0 nnsight==0.7.0
#   transformers==4.57.3 huggingface_hub==0.36.2
#   transformer_lens==3.2.1
from circuit_tracer import ReplacementModel, attribute, Graph
from circuit_tracer.utils import create_graph_files
# "gemma" = Gemma Scope transcoders for Gemma 2 2B (gated)
rm = ReplacementModel.from_pretrained(
    "google/gemma-2-2b", "gemma", dtype=torch.bfloat16,
    backend="nnsight")  # TransformerLens exhausts Colab RAM
# Experimental: from_pretrained("google/gemma-3-1b-it",
#   "mwhanna/gemma-scope-2-1b-it/transcoder_all/"
#   "width_16k_l0_small", backend="nnsight")
prompt = "Fact: the capital of the state containing Dallas is"
graph = attribute(prompt, rm, max_n_logits=10,
                  desired_logit_prob=0.95, batch_size=256,
                  max_feature_nodes=4096, verbose=True)
graph.to_pt("dallas.pt")       # Graph.from_pt("dallas.pt")
create_graph_files(graph, slug="dallas",
                   output_path="graphs/",   # pruned JSON
                   node_threshold=0.8, edge_threshold=0.98)
# validate: (layer, position, feature_idx, new_value)
logits, acts = rm.feature_intervention(
    prompt, [(20, 10, 15589, 0.0)])
```

NB4 default `PATH = "neuronpedia"`: no download. The Gemma 2 2B path is opt-in (~18 GB). Featured Dallas graph: see Links. `attribute(prompt, model, ...)`: prompt first. Rows of `graph.active_features` are (layer, pos, feature). Big *error nodes* = computation the transcoders miss. Ids above show the format only.

## Glossary

| | |
|---|---|
| **Residual stream** | The running sum every layer reads from and writes to; per-token shared memory. |
| **Logit lens** | Final LayerNorm + unembedding applied to an intermediate residual: its current guess. |
| **Activation patching** | Copy a clean-run activation into a corrupted run; measure how much behaviour returns. |
| **Logit difference** | logit(correct) − logit(alternative); a clean metric for patching. |
| **Polysemantic** | One neuron responds to several unrelated concepts. |
| **Superposition** | More features than dimensions, stored as almost-orthogonal directions. |
| **SAE / latent** | Sparse autoencoder: activation = sparse sum of decoder directions (latents). |
| **Transcoder** | Like an SAE, but reads an MLP's input and predicts its output. |
| **Attribution graph** | Direct linear effects among tokens, transcoder features, error nodes and logits for one prompt. |
| **Error node** | What the transcoders fail to reconstruct at a layer and position: unexplained computation. |
| **Probe** | A small, usually linear, classifier trained on activations to read out a property. |
| **Steering vector** | A direction added to the residual stream to push behaviour; not unique. |
| **Calibration** | Stated confidence matches observed accuracy (80% sure → right 80% of the time). |
| **Risk–coverage** | Error among accepted items versus share accepted, as the threshold moves. |
| **Escape rate** | Wrong verdicts that pass the threshold and skip human review. |

## Links

| | |
|---|---|
| **Neuronpedia** | `neuronpedia.org` |
| **Dallas graph (NB4)** | `neuronpedia.org/gemma-2-2b/graph?slug=gemma-fact-dallas-austin` |
| **Transformer Explainer** | `poloclub.github.io/transformer-explainer` |
| **ARENA, chapter 1** | `learn.arena.education` |
| **TransformerLens** | `transformerlensorg.github.io/TransformerLens` |
| **SAELens** | `decoderesearch.github.io/SAELens` |
| **circuit-tracer** | `github.com/decoderesearch/circuit-tracer` |
| **Gemma Scope 2** | `huggingface.co/collections/google/gemma-scope-2` |
| **Open problems** | `arXiv 2501.16496 (Sharkey et al.)` |


Miguel González · Salvador Hinojosa · Víctor Terrón-Macías